

 Avaliação Final — Big Data para Negócios

Curso: Big Data para Negócios

Instituição: UFC - Universidade Federal do Ceará

Professor responsável: Luiz Alexandre Moreira Barros

Formato: Individual · Relatório + repositório de código + dashboard

Aluno(a): Gabriel Ramos e Thales Vieira

Data de entrega: _____

Etapa 2 - Etapa 2 — Big Data: Execução do Pipeline -
avaliacao_final/etapa2_bigdata/

Clique duas vezes (ou pressione "Enter") para editar

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, trim, lower, year, month, hour,
    when, count, sum as spark_sum, avg
)
from pyspark.sql.types import (
    StructType, StructField, IntegerType,
    DoubleType, StringType, TimestampType,
    BooleanType
)

spark = (
    SparkSession.builder
    .appName("TechPay_Avaliacao_Final")
    .master("local[*]")
    .getOrCreate()
)

# 1. SCHEMA

schema = StructType([
    StructField("transaction_id", IntegerType(), False),
    StructField("customer_id", IntegerType(), False),
    StructField("amount", DoubleType(), True),
    StructField("transaction_type", StringType(), True),
    StructField("channel", StringType(), True),
    StructField("merchant_category", StringType(), True),
```

```

    StructField("timestamp", TimestampType(), True),
    StructField("status", StringType(), True),
    StructField("risk_score", DoubleType(), True),
    StructField("segment", StringType(), True),
    StructField("credit_score", IntegerType(), True),
    StructField("is_fraud", BooleanType(), True)
])

# 2. INGESTÃO / RAW

raw_df = (
    spark.read
    .option("header", True)
    .schema(schema)
    .csv("/content/avaliacao_transactions.csv")
)

print("RAW:", raw_df.count())

raw_df.printSchema()

# 3. BRONZE - LIMPEZA

bronze_df = (
    raw_df
    .dropDuplicates(["transaction_id"])
    .withColumn("transaction_type", lower(trim(col("transaction_type"))))
    .withColumn("channel", lower(trim(col("channel"))))
    .withColumn("merchant_category", lower(trim(col("merchant_category"))))
    .withColumn("status", lower(trim(col("status"))))
)

bronze_df = bronze_df.filter(
    (col("transaction_id") > 0) &
    (col("customer_id") >= 1) &
    (col("customer_id") <= 8000) &
    (col("amount") > 0) &
    (col("risk_score") >= 0) &
    (col("risk_score") <= 100) &
    (col("credit_score") >= 300) &
    (col("credit_score") <= 900) &
    (col("timestamp").isNotNull())
)

print("BRONZE:", bronze_df.count())

# Particionamento por ano/mês
bronze_df = (
    bronze_df
    .repartition(12, col("timestamp").yearMonth())
    .sortWithinPartitions(col("timestamp"))
)

```

```
bronze_df
.withColumn("ano", year("timestamp"))
.withColumn("mes", month("timestamp"))
)

(
    bronze_df.write
    .mode("overwrite")
    .partitionBy("ano", "mes")
    .parquet("data/bronze")
)

# 4. SILVER – ENRIQUECIMENTO

silver_df = (
    bronze_df
    .withColumn(
        "faixa_valor",
        when(col("amount") < 50, "ate_50")
        .when(col("amount") < 200, "50_199")
        .when(col("amount") < 500, "200_499")
        .when(col("amount") < 1000, "500_999")
        .otherwise("1000_mais")
    )
    .withColumn("hora", hour("timestamp"))
    .withColumn(
        "periodo_dia",
        when(col("hora").between(0, 5), "madrugada")
        .when(col("hora").between(6, 11), "manha")
        .when(col("hora").between(12, 17), "tarde")
        .otherwise("noite")
    )
    .withColumn(
        "faixa_risco",
        when(col("risk_score") < 30, "baixo")
        .when(col("risk_score") < 70, "medio")
        .otherwise("alto")
    )
)

print("SILVER:", silver_df.count())

(
    silver_df.write
    .mode("overwrite")
    .partitionBy("ano", "mes")
    .parquet("data/silver")
)
```

5. GOLD 1 – FRAUDE POR CANAL

```
gold_canal = (
    silver_df
    .groupBy("channel")
    .agg(
        count("*").alias("total_transacoes"),
        spark_sum(
            when(col("is_fraud") == True, 1).otherwise(0)
        ).alias("total_fraudes"),
        spark_sum("amount").alias("valor_total"),
        avg("risk_score").alias("risco_medio")
    )
    .withColumn(
        "taxa_fraude",
        col("total_fraudes") / col("total_transacoes")
    )
)

gold_canal.show(truncate=False)

gold_canal.write.mode("overwrite").parquet(
    "data/gold/fraude_por_canal"
)
```

6. GOLD 2 – FRAUDE POR CATEGORIA

```
gold_categoria = (
    silver_df
    .groupBy("merchant_category")
    .agg(
        count("*").alias("total_transacoes"),
        spark_sum(
            when(col("is_fraud") == True, 1).otherwise(0)
        ).alias("total_fraudes"),
        avg("risk_score").alias("risco_medio"),
        avg("amount").alias("ticket_medio")
    )
    .withColumn(
        "taxa_fraude",
        col("total_fraudes") / col("total_transacoes")
    )
)

gold_categoria.show(truncate=False)

gold_categoria.write.mode("overwrite").parquet(
    "data/gold/fraude_por_categoria"
)
```

```
)
spark.stop()
```

RAW: 30000

root

```
-- transaction_id: integer (nullable = true)
-- customer_id: integer (nullable = true)
-- amount: double (nullable = true)
-- transaction_type: string (nullable = true)
-- channel: string (nullable = true)
-- merchant_category: string (nullable = true)
-- timestamp: timestamp (nullable = true)
-- status: string (nullable = true)
-- risk_score: double (nullable = true)
-- segment: string (nullable = true)
-- credit_score: integer (nullable = true)
-- is_fraud: boolean (nullable = true)
```

BRONZE: 30000

SILVER: 30000

	channel	total_transacoes	total_fraudes	valor_total	risco_mec
pos	5968	85	1088597.6699999978	50.593396	
atm	3007	54	538079.5699999998	49.502440	
app	12021	455	2007907.0599999952	49.822696	
web	9004	157	1573259.1999999958	50.364439	

	merchant_category	total_transacoes	total_fraudes	risco_medio
servicos	4576	90	49.92248907342647	
saude	2951	73	50.02344967807517	
viagem	3010	160	50.82484717607971	
varejo	9047	203	50.312765557643445	
alimentacao	5976	138	49.685634203480475	
eletronico	4440	87	50.01059459459453	

